Experiment No. 3
Creating a Cryptocurrency (Hadcoin) using Python and Mining in a Blockchain

Aim

To create a cryptocurrency using Python and perform mining in the blockchain created, using a simulated peer-to-peer network of three nodes.

Course Outcomes: CO1, CO2, CO5

Theory

Blockchain is a distributed ledger of linked blocks. Each block holds transactions, a timestamp, the hash of the previous block and its own hash, so changing one block invalidates all later blocks. **Mining** collects pending transactions into a block, solves a Proof-of-Work puzzle (a hash starting with `0000`), adds the block to the chain and rewards the miner with a transaction.

Transactions and mempool – a transaction (sender, receiver, amount) sent to `/add_transaction` waits in the mempool, the pool of pending transactions. When a block is mined, these transactions are placed in the block and removed from the mempool.

Consensus – every node keeps its own copy of the chain. With the Longest Chain Rule, `/replace_chain` makes a node fetch the chains of its peers and replace its own with the longest valid one.

Challenges in P2P networks:

- Keeping all nodes consistent (consensus and forks).
- Security – malicious nodes and double spending.
- Network latency and node failures.
- Scalability – more nodes mean more traffic and slower agreement.

Tools and libraries:

Name	Use
Flask	Web framework used to create the API endpoints of each node
requests	HTTP calls between nodes (fetching peers' chains)
Postman	Sending POST/GET requests to the nodes
datetime	Timestamp of each block
hashlib	SHA-256 hashing for block hash and Proof-of-Work
jsonify, request	Flask: JSON responses and reading request data
uuid4	Unique address of each node
urlparse	Extracting host and port of a peer's URL

Requirements

- Python 3.x with the virtual environment created in Lab 2.

- Flask and requests (`requests==2.18.4`).
- Postman.
- Files `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py` (Lab_3 folder).

Procedure

Step 1: Install the packages

Activate the virtual environment and install Flask and requests.

Step 2: Run the three nodes

Run the three files in three separate terminals. Each node runs on its own port (5001, 5002, 5003).

Step 3: Connect the nodes

In Postman, send a POST request to `/connect_node` of each node with the list of all three nodes (Figures 1–3).

Step 4: Add a transaction

Send a POST request to `/add_transaction` of node 5003 with sender, receiver and amount (Figure 4).

Step 5: Mine a block

Send a GET request to `/mine_block` of node 5003. Node 5003 now has a longer chain than 5001 and 5002 (Figure 5).

Step 6: Replace the chain

Send a GET request to `/replace_chain` of nodes 5001 and 5002 so that they take the longest chain (Figures 6–7).

Step 7: Verify the chain

Send a GET request to `/get_chain` of every node and compare the chains (Figures 8–10).

Commands Executed

```
pip install Flask
pip install requests==2.18.4

python hadcoin_node_5001.py # terminal 1
python hadcoin_node_5002.py # terminal 2
python hadcoin_node_5003.py # terminal 3
```

Request body used in `/connect_node` (same on all three nodes):

```
{ "nodes": ["http://127.0.0.1:5001",
            "http://127.0.0.1:5002",
            "http://127.0.0.1:5003"] }
```

Request body used in `/add_transaction`:

```
{ "sender": "a", "receiver": "b", "amount": 5 }
```

Program

Code of `hadcoin_node_5003.py` (the node on which the transaction was added and the block was mined). The files for 5001 and 5002 are the same except for the port in the last line (`port = 5001` / `5002`) and the receiver of the mining reward (Richard on 5001, you on 5002, Abcde on 5003). In `create_block()`, the line `self.transactions = []` removes the transactions from the mempool once they are added to the block.

```
# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4

# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4 # Generate a unique id that is in hex
from urllib.parse import urlparse # To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = [] # Adding transactions before they are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set() # Set is used as there is no order to be maintained as the nodes can be
     from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions} # Adding transactions to make the blockchain a
     cryptocurrency
        self.transactions = [] # The list of transacction should become empty after they are added to
      a block
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
```

```
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest
    ()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True

    # This method will add the trqnsaction to the list of trnsactions
    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({'sender': sender,
                                  'receiver': receiver,
                                  'amount': amount})
        previous_block = self.get_previous_block()
        return previous_block['index'] + 1 # It will return the block index to which the transaction
    should be added

    # This function will add the node containing an address to the set of nodes created in init
    function
    def add_node(self, address):
        parsed_url = urlparse(address) # urlparse will parse the url from the address
        self.nodes.add(parsed_url.netloc) # Add is used and not append as it's a set. Netloc will
    only return '127.0.0.1:5000'

    # Consensus Protocol. This function will replace all the shorter chain with the longer chain in
    all the nodes on the network
    def replace_chain(self):
        network = self.nodes # network variable is the set of nodes all around the globe
        longest_chain = None # It will hold the longest chain when we scan the network
        max_length = len(self.chain) # This will hold the length of the chain held by the node that
    runs this function
        for node in network:
            response = requests.get(f'http://{node}/get_chain') # Use get chain method already
    created to get the length of the chain
            if response.status_code == 200:
                length = response.json()['length'] # Extract the length of the chain from get_chain
    fiunction
                chain = response.json()['chain']
                if length > max_length and self.is_chain_valid(chain): # We check if the length is
    bigger and if the chain is valid then
                    max_length = length # We update the max length
                    longest_chain = chain # We update the longest chain
        if longest_chain: # If longest_chain is not none that means it was replaced
            self.chain = longest_chain # Replace the chain of the current node with the longest chain
            return True
        return False # Return false if current chain is the longest one

# Part 2 - Mining our Blockchain

# Creating a Web App
app = Flask(__name__)

# Creating an address for the node on Port 5000. We will create some other nodes as well on different
     ports
node_address = str(uuid4()).replace('-', '') #

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
```

```
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Abcde', amount = 1) # Hadcoins to
     mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the Blockchain
@app.route('/add_transaction', methods = ['POST']) # Post method as we have to pass something to get
     something in return
def add_transaction():
    json = request.get_json() # This will get the json file from postman. In Postman we will create a
      json file in which we will pass the values for the keys in the json file
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys): # Checking if all keys are available in json
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201 # Code 201 for creation

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes
@app.route('/connect_node', methods = ['POST']) # POST request to register the new nodes from the
     json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes') # Get the nodes from json file
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the
     following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

# Replacing the chain by the longest chain if needed
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
     longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5003)
```

Output

Connecting the nodes

All three nodes were connected; each node lists the three peers in `total_nodes`.

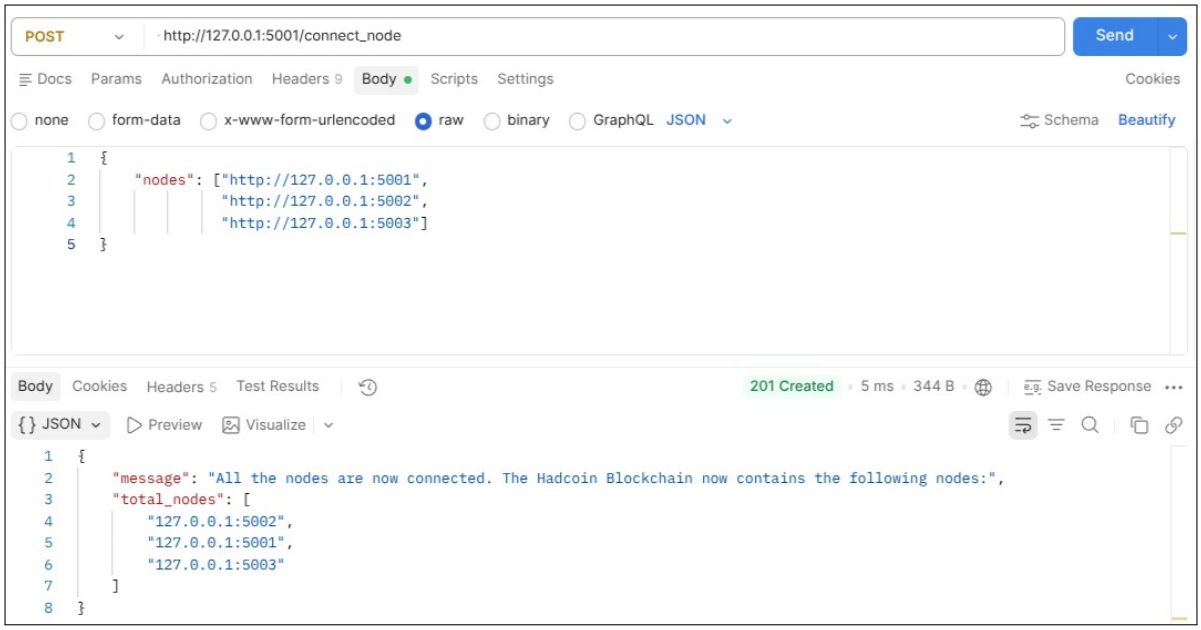


Figure 1: POST /connect_node on node 5001

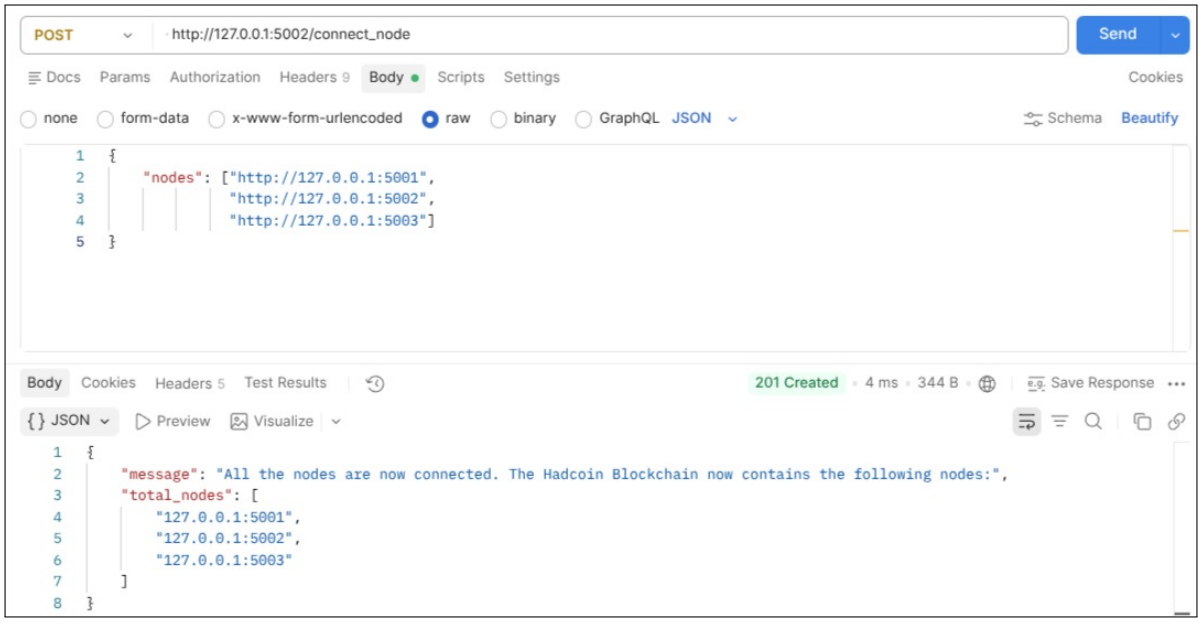


Figure 2: POST /connect_node on node 5002

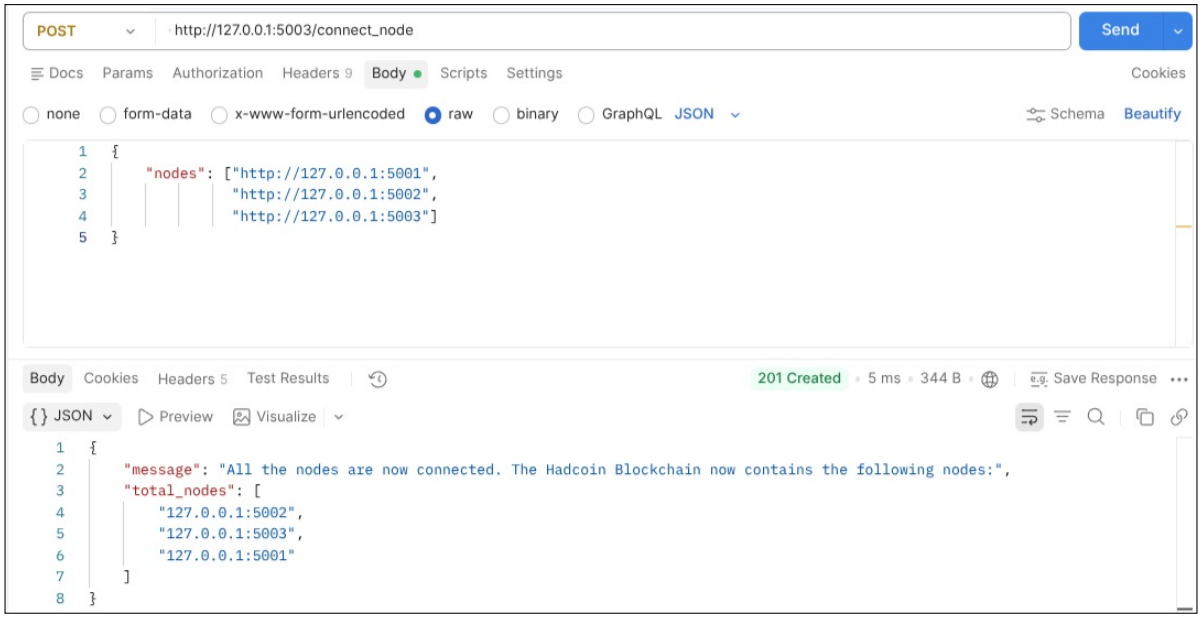


Figure 3: POST /connect_node on node 5003

Adding a transaction

The transaction a → b of 5 was accepted by node 5003 and will be added to Block 2.



Figure 4: POST /add_transaction on node 5003

Mining

Node 5003 mined Block 2 (proof 533). The block contains the transaction a → b and the mining reward of 1 coin.

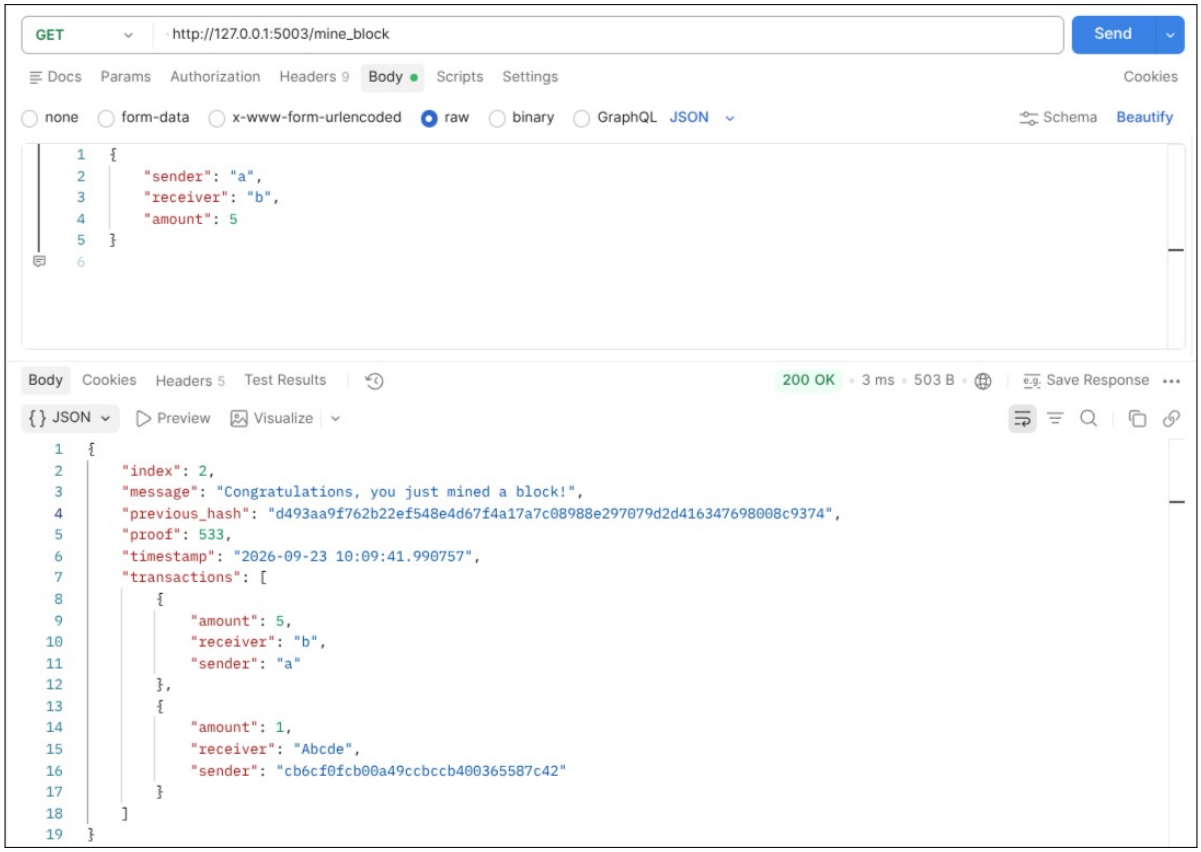


Figure 5: GET /mine_block on node 5003

Replacing the chain

Nodes 5001 and 5002 had shorter chains, so they replaced them with the longest chain.

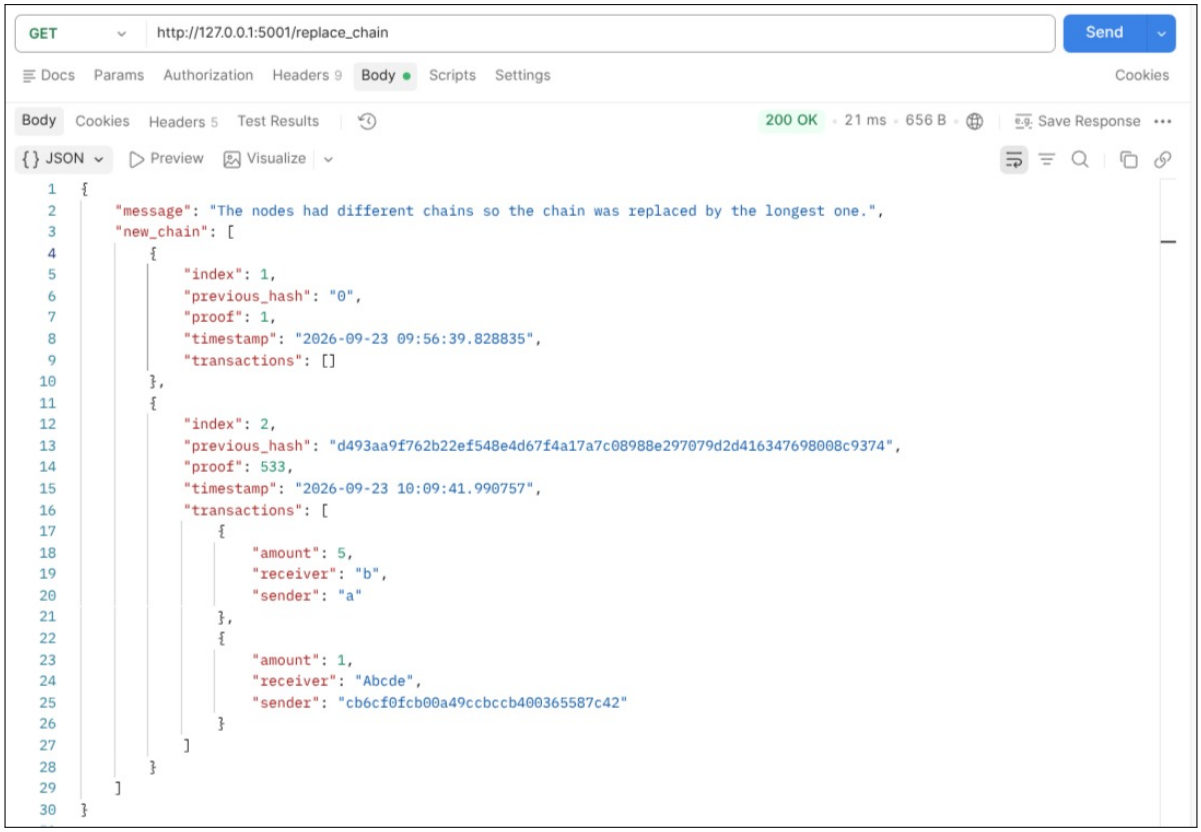


Figure 6: GET /replace_chain on node 5001

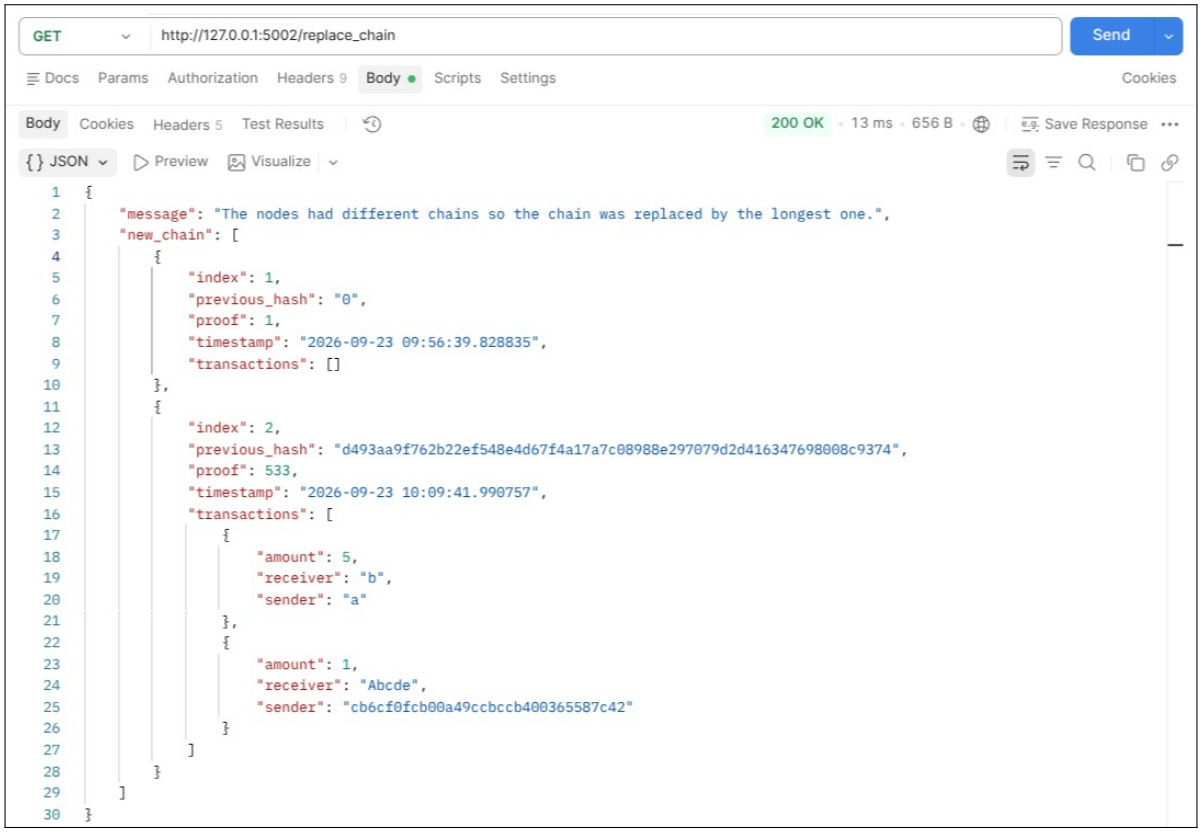


Figure 7: GET /replace_chain on node 5002

Fetching the chain

All three nodes now return the same chain of length 2, with the same hashes and transactions.

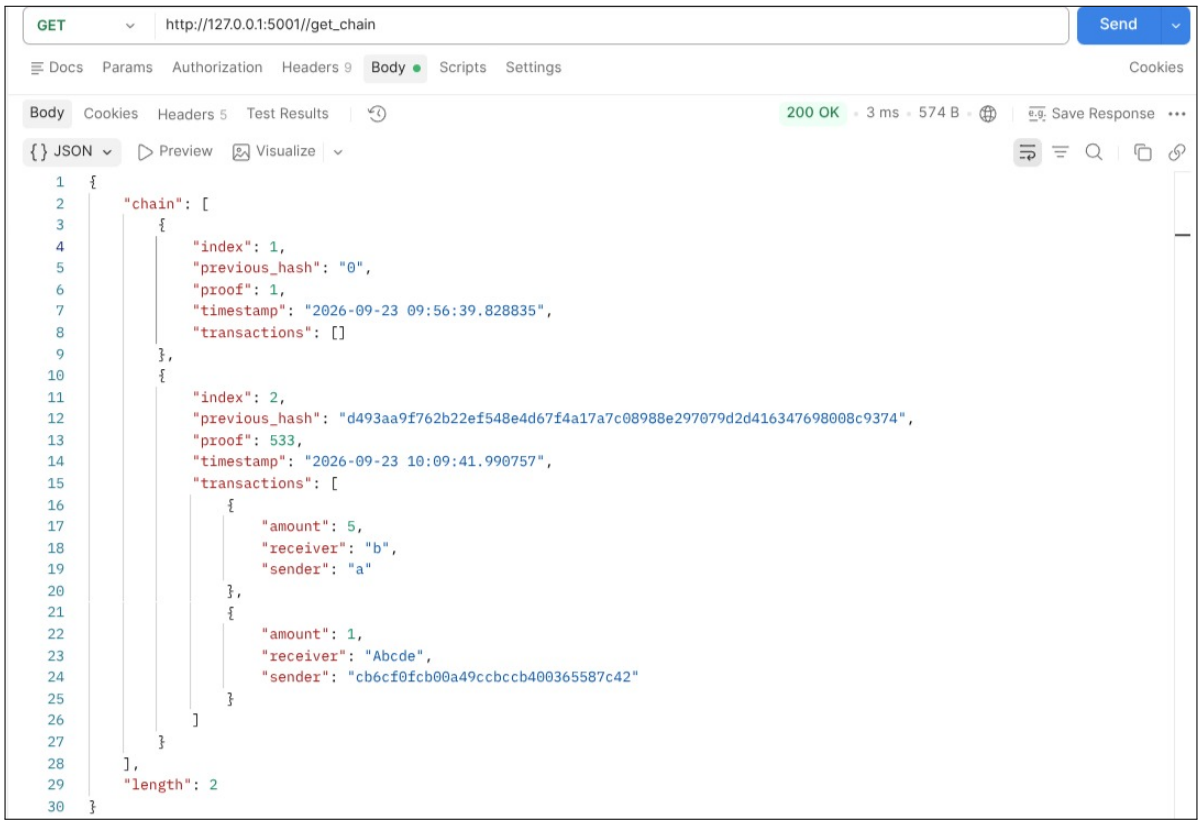


Figure 8: GET /get_chain on node 5001

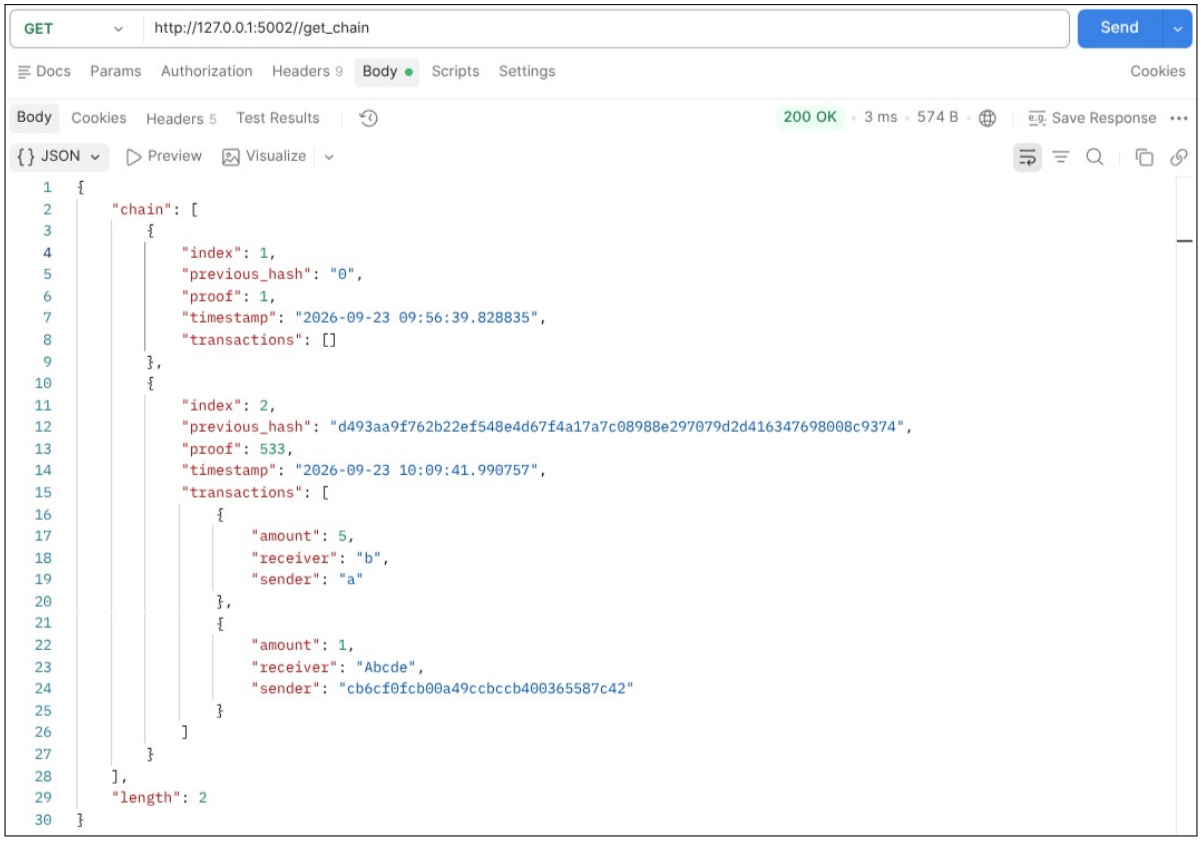


Figure 9: GET /get_chain on node 5002

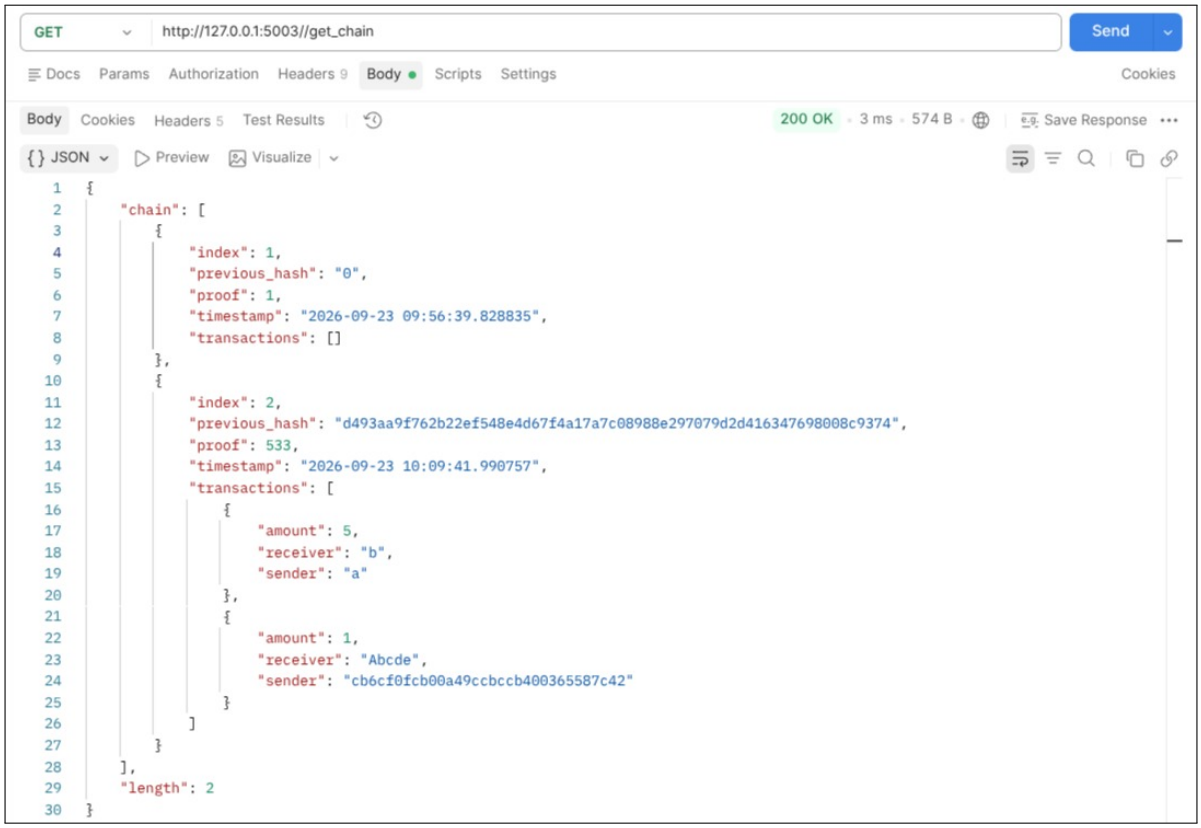


Figure 10: GET /get_chain on node 5003

Conclusion

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.

Experiment completed successfully.